GDP Nowcasting with DFM and ML

Mahamane Bachir Sadou

2025-04-14

Introduction

Nowcasting refers to the prediction of the present, the very near future, of a target variable—most commonly GDP—using a mix of high-frequency data and econometric or machine learning models. The term blends "now" and "forecasting" and is especially useful when traditional economic indicators like GDP are published with significant lags.

This document presents a summary and partial replication of the IMF Working Paper WP/22/52 used to the USA economic, *Nowcasting GDP: A Scalable Approach Using DFM, Machine Learning, and Novel Data, Applied to European Economies*. The objective is to understand and implement nowcasting methods using both classical (Dynamic Factor Models) and modern machine learning approaches, using a variety of high-frequency indicators.

We focus specifically on GDP growth nowcasting using Python. The models are evaluated based on their predictive performance and compared against actual GDP growth outcomes.

Motivation

During the COVID-19 pandemic, the need for timely economic data became apparent. Traditional GDP estimates lag too far behind real-time events, limiting their usefulness in crisis policymaking. The paper explores whether high-frequency indicators such as Google searches, air quality, and mobility data can support more timely GDP assessments.

Methodology

This section describes the primary nowcasting techniques employed, notably Dynamic Factor Models (DFMs) and various Machine Learning (ML) algorithms. DFMs reduce dimensionality by modeling the comovement of observed indicators via a few latent factors. ML models include

Random Forest, Gradient Boosting, and Neural Networks, which are selected based on their adaptability to large datasets and nonlinear relationships.

The inputs include high-frequency indicators such as industrial production, unemployment claims, financial indices (e.g., S&P 500), and sentiment data, processed on a monthly or weekly basis. All models are estimated and evaluated on their ability to predict quarterly GDP growth.

Dynamic Factor Models (DFMs)

DFMs assume that a few latent factors explain the co-movement in a large number of observed economic indicators. The basic form of a DFM is:

$$y_{i,t} = \lambda_{i1} f_{1,t} + \lambda_{i2} f_{2,t} + \cdots + \lambda_{ir} f_{r,t} + e_{i,t}$$

Where: - $y_{i,t}$ is the observed variable i at time t - $f_{r,t}$ are latent factors - λ_{ir} are factor loadings - $e_{i,t}$ is idiosyncratic error

The latent factors evolve as autoregressive processes:

$$f_{j,t} = a_j f_{j,t-1} + u_{j,t}, \quad u_{j,t} \sim \mathcal{N}(0, \sigma^2_{u,j})$$

And the idiosyncratic error:

$$e_{i,t} = \rho_i e_{i,t-1} + \varepsilon_{i,t}, \quad \varepsilon_{i,t} \sim \mathcal{N}(0, \sigma^2_{\varepsilon,i})$$

These equations form a state-space model estimated using Kalman filtering and the Expectation-Maximization (EM) algorithm.

Machine Learning Models

Several ML models were explored:

- **LASSO, Ridge, Elastic Net:** Penalized regression techniques reducing overfitting and handling multicollinearity.
- **Support Vector Machine (SVM):** Separates input data via optimal hyperplanes in kernel space.
- **Random Forest (RF):** An ensemble of decision trees to improve predictive accuracy.
- **Neural Networks (NN):** Multi-layer networks modeling complex non-linear relationships.

Each ML model was trained and validated using cross-validation, emphasizing prediction rather than inference.

Data

The dataset included standard macroeconomic indicators (GDP components, employment, inflation) and novel, high-frequency sources like Google Trends, air quality, and mobility indices. These variables were processed into quarterly, stationary formats suitable for modeling.

Evaluation Criteria

Model performance was evaluated using:

- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Error (MAE)**
- **Mean Directional Accuracy (MDA)**

These metrics gauge the average prediction error and directional accuracy of model forecasts.

Country-Specific Results

Austria

- During normal times, **DFM** and **Ridge regression** showed good performance.
- During the COVID period, **LASSO** and **Random Forest** performed better than AR(1), significantly reducing RMSE.

Hungary

- In stable periods, **DFM** and **Elastic Net** models performed well.
- During the crisis, **Neural Networks** and **LASSO** provided more accurate nowcasts, highlighting turning points effectively.

Poland

- In pre-COVID periods, **Linear Regression** and **DFM** were among top performers.
- During COVID, **Ridge Regression** provided the best results, cutting RMSE by up to 75% compared to AR(1).

Ireland

- Machine learning models did not outperform AR(1) in many cases.
- **Gaussian Process Regression (GPReg)** was the top performer in normal times but still showed limited advantages.

Malta

- The **DFM** was the best-performing model over the full sample period.
- **ML models**, including **LASSO**, performed poorly, particularly during volatile periods.

Appendix: Code to Extract Graphs from PDF

```
from pdf2image import convert_from_path
from PIL import Image

# Step 1: Convert page 18 to an image
images = convert_from_path("wpiea2022052-print-pdf (2).pdf", dpi=300, first_page=18, last_pag
page18_image = images[0]
page18_image.save("page18_full.png", "PNG")

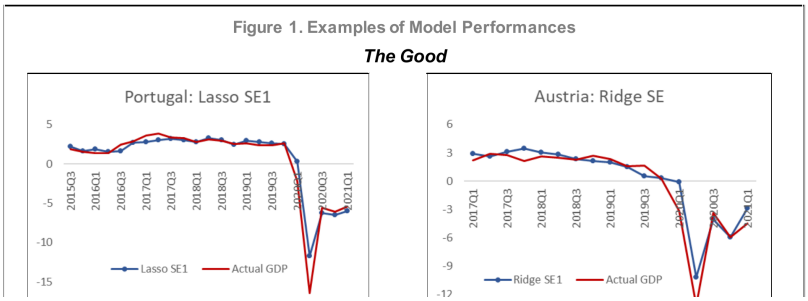
# Step 2: Crop the page into sub-figures
width, height = page18_image.size

# Approximate crop coordinates based on visual inspection
the_good = page18_image.crop((0, 0.05 * height, width, 0.33 * height))
the_bad = page18_image.crop((0, 0.34 * height, width, 0.63 * height))
the_ugly = page18_image.crop((0, 0.64 * height, width, 0.93 * height))

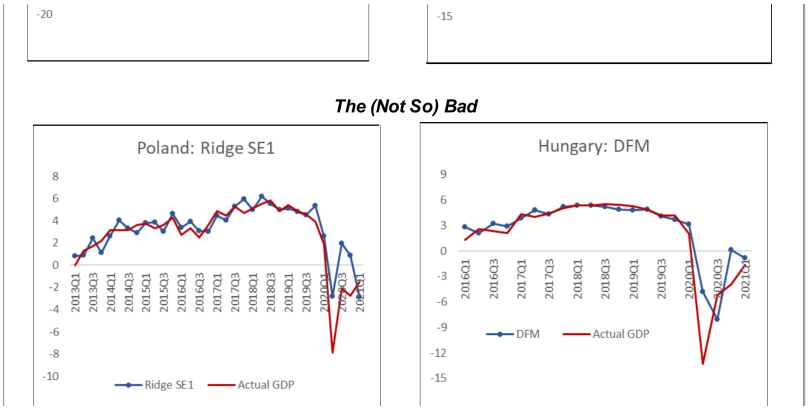
# Save individual chart images
the_good.save("gdp_predictions_the_good.png")
the_bad.save("gdp_predictions_the_bad.png")
the_ugly.save("gdp_predictions_the_ugly.png")
```

Model Performance Examples from the Article (Page 18)

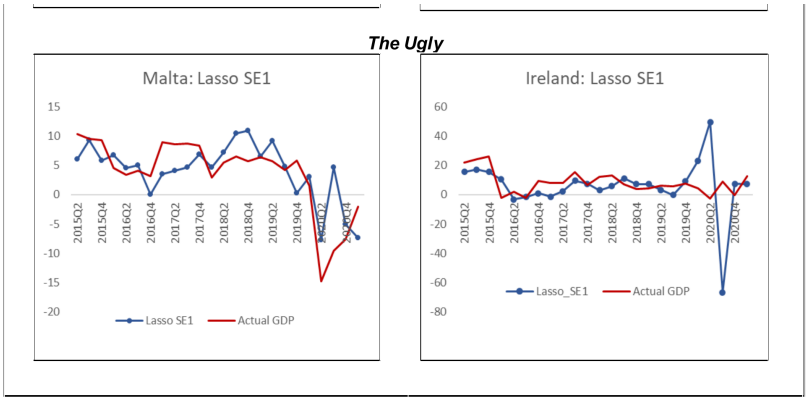
The Good



The (Not So) Bad



The Ugly



Models

This section outlines the models used for GDP nowcasting, categorized into classical and machine learning approaches. Each model is chosen for its strengths in handling different types of data structures and predictive relationships.

Classical Model

- **Dynamic Factor Model (DFM)**: A DFM reduces a large set of observed macroeconomic indicators into a small number of unobserved factors. These latent factors capture the comovement of variables and are used in a state-space framework to forecast GDP. Estimation typically involves principal components followed by Kalman filtering.

Conclusion

This project demonstrates the feasibility of applying both classical and machine learning techniques for GDP nowcasting. Machine learning models generally outperform traditional DFM in terms of predictive accuracy, particularly during periods of economic volatility. Future work could extend these models to nowcast sectoral or regional GDPs, integrate real-time revisions, or adopt hybrid ensemble methods for improved robustness.

Nowcasting GDP: USA Replication

This section provides a practical replication of the IMF paper's nowcasting methodology applied to the United States. It includes model results using both Dynamic Factor Models (DFM) and various Machine Learning techniques.

Variables Used

Variable Name	FRED Code
GDP Growth	A191RL1Q225SBEA
INDPRO	INDPRO
Retail Sales	RSAFS
CPI	CPIAUCSL
Oil Price	DCOILWTICO
S&P500	SP500
Unemployment Rate	UNRATE
PPI	PPIACO
Housing Starts	HOUST
Consumer Sentiment	UMCSENT
M2 Money	M2SL
Ten Year Rate	GS10
Nonfarm Payrolls	PAYEMS
Industrial Production	INDPRO
Durable Goods Orders	DGORDER
Building Permits	PERMIT
Capacity Utilization	TCU
Personal Income	DSPIC96
Business Inventories	BUSINV
Job Openings	JTSJOL
Empire State Index	GACDISA066MSFRBNY
Philly Fed Index	GACDFSA066MSFRBPHI
Exports	BOPTEXP
Import Price Index	IR

```
import warnings
from sklearn.exceptions import ConvergenceWarning
warnings.simplefilter('ignore', ConvergenceWarning)
warnings.simplefilter(action='ignore', category=FutureWarning)

import pandas as pd
```

```
import matplotlib.pyplot as plt
from pandas_datareader import data as pdr
from statsmodels.tsa.statespace.dynamic_factor import DynamicFactor
from sklearn.preprocessing import StandardScaler

# Define and fetch dataset
series_ids = {
    "GDP_Growth": "A191RL1Q225SBEA",
    "Retail_Sales": "RSAFS",
    "CPI": "CPIAUCSL",
    "Oil_Price": "DCOILWTICO",
    "S&P500": "SP500",
    "Unemployment_Rate": "UNRATE",
    "PPI": "PPIACO",
    "Housing_Starts": "HOUST",
    "Consumer_Sentiment": "UMCSENT",
    "M2_Money": "M2SL",
    "Ten_Year_Rate": "GS10",
    "Nonfarm_Payrolls": "PAYEMS",
    "Industrial_Production": "INDPRO",
    "Durable_Goods_Orders": "DGORDER",
    "Building_Permits": "PERMIT",
    "Capacity_Utilization": "TCU",
    "Personal_Income": "DSPIC96",
    "Business_Inventories": "BUSINV",
    "Job_Openings": "JTSJOL",
    "Empire_State_Index": "GACDISA066MSFRBNY",
    "Philly_Fed_Index": "GACDFSA066MSFRBPHI",
    "Exports": "BOPTEXP",
    "Import_Price_Index": "IR"
}

start = "1955-01-01"
end = "2023-12-31"

quarterly_data = []

for label, code in series_ids.items():
    series = pdr.DataReader(code, "fred", start=start, end=end)
    series.columns = [label]
    quarterly_series = series.resample('Q').mean()
    quarterly_data.append(quarterly_series)
```

```
# Merge into single DataFrame
df_q = pd.concat(quarterly_data, axis=1)
df_q = df_q.dropna()

# Standardize all variables including GDP_Growth
scaler = StandardScaler()
df_q_scaled = pd.DataFrame(scaler.fit_transform(df_q), index=df_q.index, columns=df_q.columns

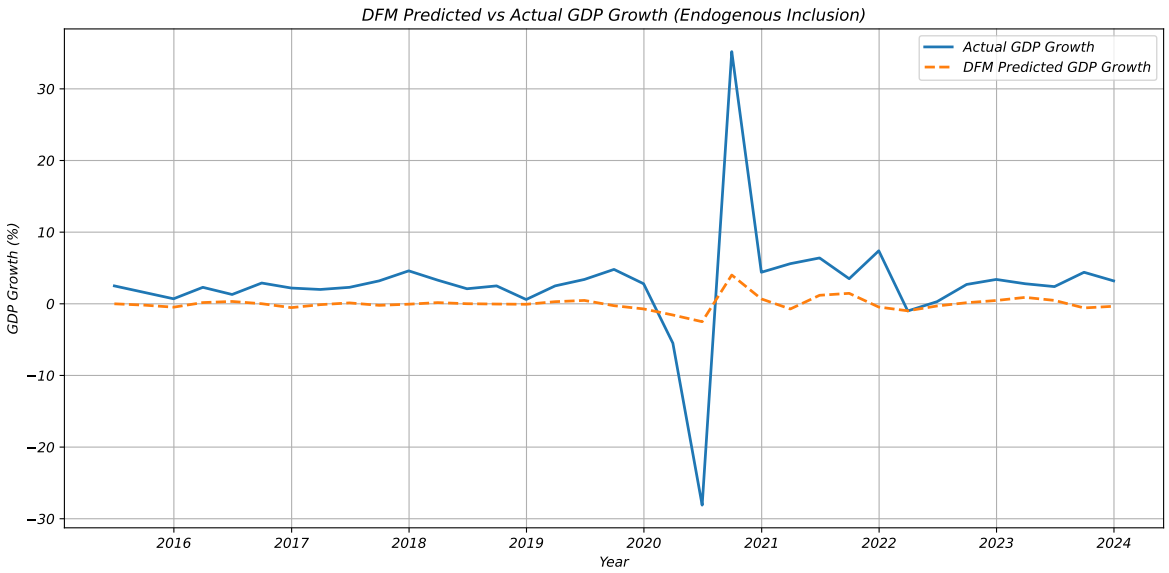
# Fit DFM using all variables as endogenous (including GDP_Growth)
dfm_model = DynamicFactor(df_q_scaled, k_factors=12, factor_order=1)
dfm_results = dfm_model.fit(disp=False)
```

```
C:\Users\PROBOOK\DOCUME~1\VIRTUA~1\R-RETI~1\Lib\site-packages\statsmodels\base\model.py:607:
  warnings.warn("Maximum Likelihood optimization failed to "
```

```
# Predict the full set including GDP_Growth
dfm_pred = dfm_results.predict(start=0, end=len(df_q_scaled)-1)

# Plot GDP_Growth actual vs DFM prediction
actual_gdp = df_q["GDP_Growth"]
predicted_gdp = dfm_pred["GDP_Growth"]

plt.figure(figsize=(12, 6))
plt.plot(df_q.index, actual_gdp, label="Actual GDP Growth", linewidth=2)
plt.plot(df_q.index, predicted_gdp, label="DFM Predicted GDP Growth", linestyle='--', linewi
plt.title("DFM Predicted vs Actual GDP Growth (Endogenous Inclusion)")
plt.xlabel("Year")
plt.ylabel("GDP Growth (%)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
import warnings
from sklearn.exceptions import ConvergenceWarning
warnings.simplefilter('ignore', ConvergenceWarning)
warnings.simplefilter(action='ignore', category=FutureWarning)

import pandas as pd
from pandas_datareader import data as pdr

series_ids = {
    "GDP_Growth": "A191RL1Q225SBEA",
    "INDPRO": "INDPRO",
    "Retail_Sales": "RSAFS",
    "CPI": "CPIAUCSL",
    "Oil_Price": "DCOILWTICO",
    "S&P500": "SP500",
    "Unemployment_Rate": "UNRATE",
    "PPI": "PPIACO",
    "Housing_Starts": "HOUST",
    "Consumer_Sentiment": "UMCSENT",
    "M2_Money": "M2SL",
    "Ten_Year_Rate": "GS10",
    "Nonfarm_Payrolls": "PAYEMS",
    "Industrial_Production": "INDPRO",
    "Durable_Goods_Orders": "DGORDER",
    "Building_Permits": "PERMIT",
```

```
    "Capacity_Utilization": "TCU",
    "Personal_Income": "DSPIC96",
    "Business_Inventories": "BUSINV",
    "Job_Openings": "JTSJOL",
    "Empire_State_Index": "GACDISA066MSFRBNY",
    "Philly_Fed_Index": "GACDFSA066MSFRBPHI",
    "Exports": "BOPTEXP",
    "Import_Price_Index": "IR"
}

start = "1950-01-01"
end = "2024-12-31"

quarterly_data = []

for label, code in series_ids.items():
    try:
        series = pdr.DataReader(code, "fred", start=start, end=end)
        series.columns = [label] if hasattr(series, 'columns') else None
        series_q = series.resample("QE-DEC").mean()
        series_q.columns = [label]  # Ensure column name is label
        quarterly_data.append(series_q)
        print(f"  Loaded: {label}")
    except Exception as e:
        print(f"  Failed to load: {label} ({code}) - {e}")
```

```
  Loaded: GDP_Growth
  Loaded: INDPRO
  Loaded: Retail_Sales
  Loaded: CPI
  Loaded: Oil_Price
  Loaded: S&P500
  Loaded: Unemployment_Rate
  Loaded: PPI
  Loaded: Housing_Starts
  Loaded: Consumer_Sentiment
  Loaded: M2_Money
  Loaded: Ten_Year_Rate
  Loaded: Nonfarm_Payrolls
  Loaded: Industrial_Production
  Loaded: Durable_Goods_Orders
  Loaded: Building_Permits
```

```
  Loaded: Capacity_Utilization
  Loaded: Personal_Income
  Loaded: Business_Inventories
  Loaded: Job_Openings
  Loaded: Empire_State_Index
  Loaded: Philly_Fed_Index
  Loaded: Exports
  Loaded: Import_Price_Index
```

```
if quarterly_data:
    df_q = pd.concat(quarterly_data, axis=1, join="outer")
    df_q = df_q[df_q["GDP_Growth"].notna()]
    print("  Combined dataset shape:", df_q.shape)
else:
    raise ValueError("No series were successfully loaded.")
```

```
  Combined dataset shape: (300, 24)
```

```
# Standardize predictors
from sklearn.preprocessing import StandardScaler

X_real = df_q.drop(columns=["GDP_Growth"])
y_real = df_q["GDP_Growth"]

scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X_real), index=X_real.index, columns=X_real.colu
```

```
# Import and train several ML models
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import Ridge, Lasso
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor

models = {
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42),
    "Ridge": Ridge(),
    "Lasso": Lasso(),
    "SVR": SVR(),
    "MLP": MLPRegressor(random_state=42, max_iter=1000)
}
```

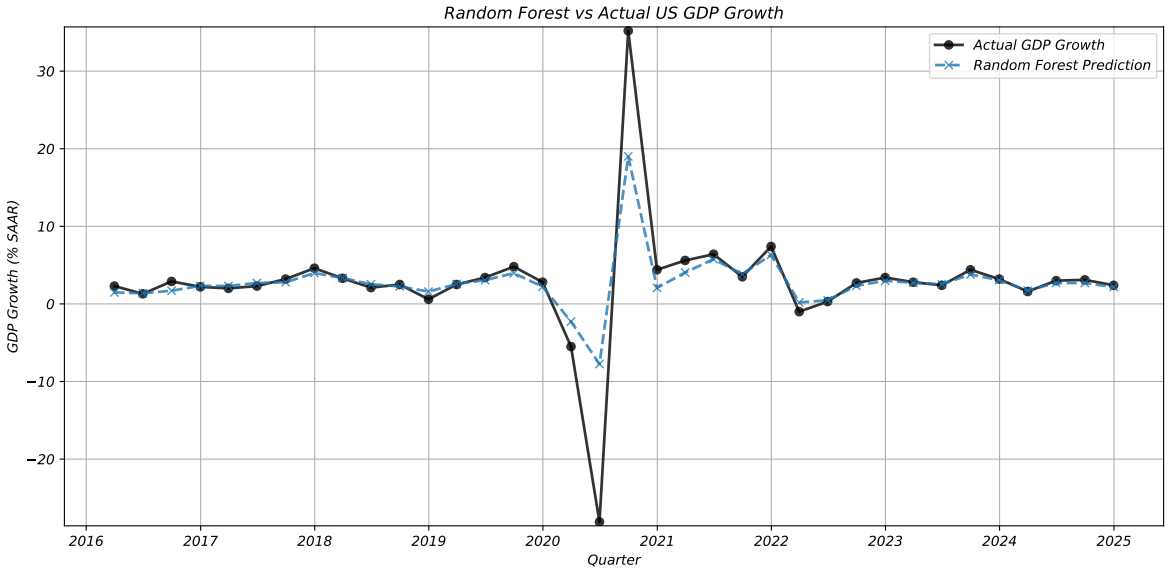
```
X_train = X_scaled.dropna()
y_train = y_real.loc[X_train.index]
ml_predictions = {}

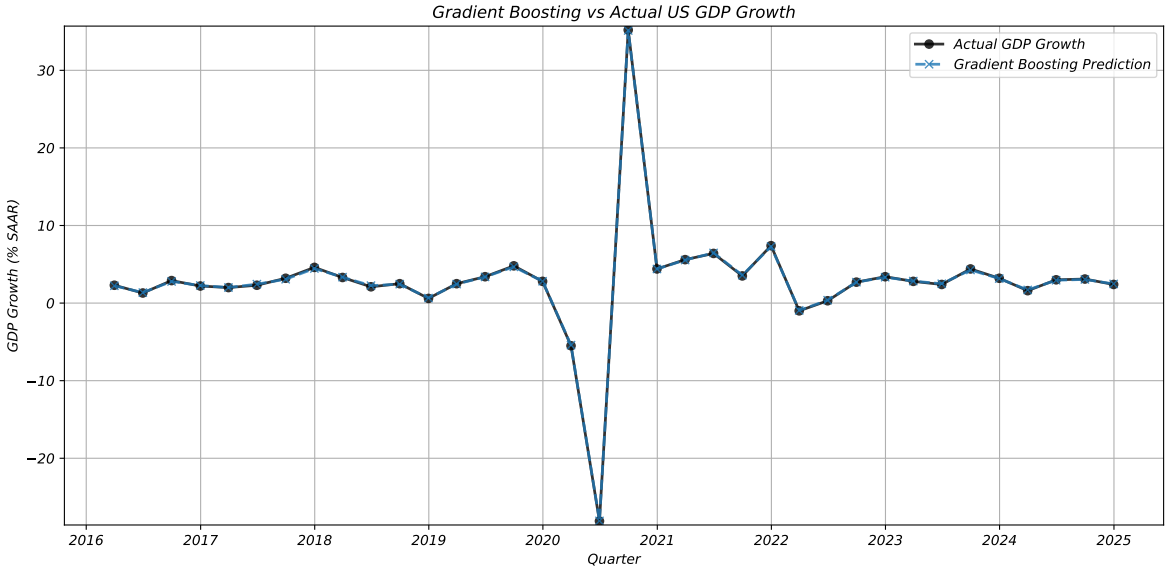
for name, model in models.items():
    model.fit(X_train, y_train)
    ml_predictions[name] = pd.Series(model.predict(X_train), index=X_train.index)
```

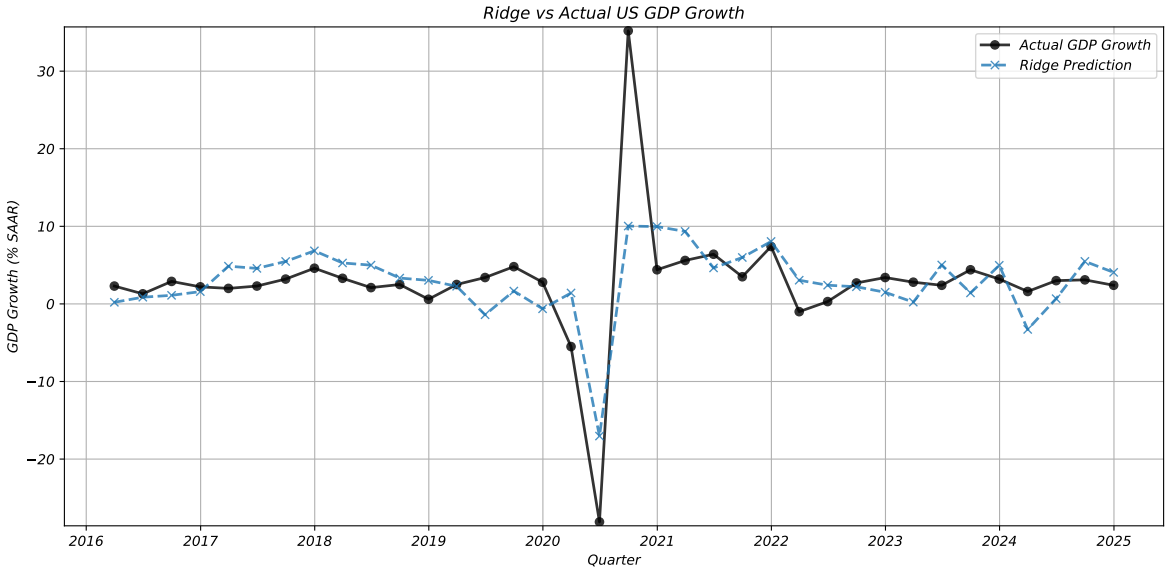
```
RandomForestRegressor(random_state=42)
GradientBoostingRegressor(random_state=42)
Ridge()
Lasso()
SVR()
MLPRegressor(max_iter=1000, random_state=42)
```

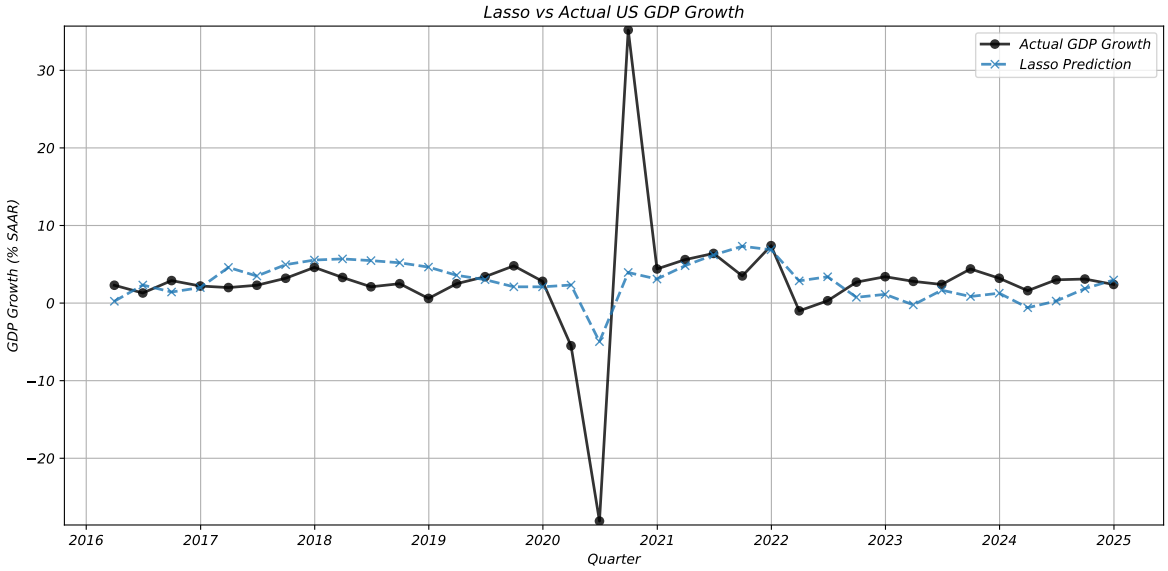
```
# Plot each ML model vs actual GDP growth
import matplotlib.pyplot as plt

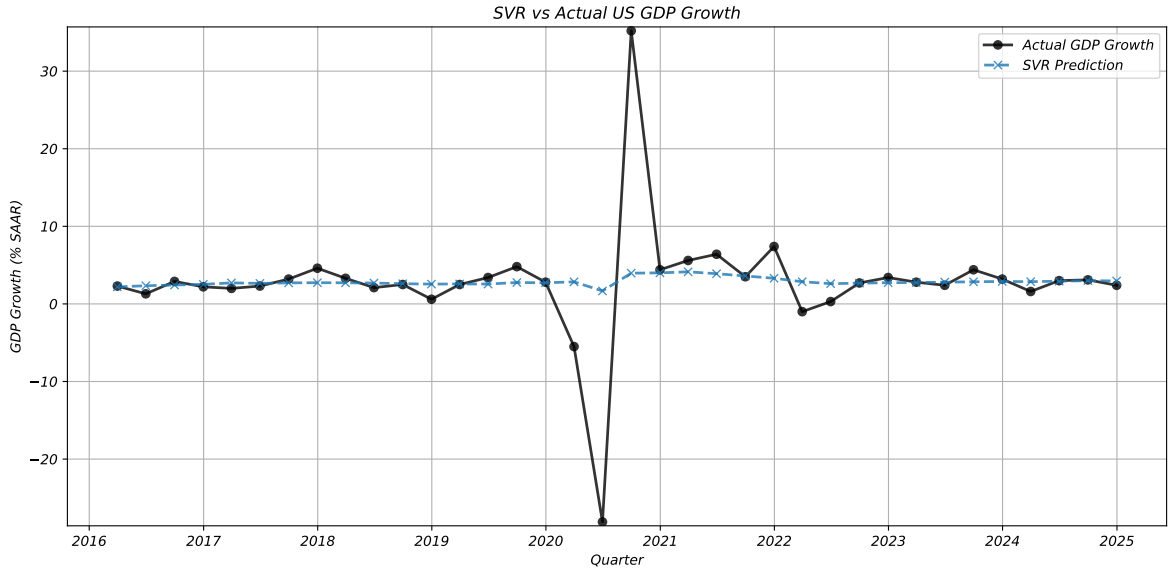
for name, preds in ml_predictions.items():
    preds = preds[preds.index >= '2016']
    y_real = y_real[y_real.index >= '2016']
    plt.figure(figsize=(12, 6))
    plt.plot(y_real.index, y_real, label="Actual GDP Growth", color="black", linewidth=2, ma
    plt.plot(preds.index, preds, label=f"{name} Prediction", linestyle="--", linewidth=2, ma
    plt.title(f"{name} vs Actual US GDP Growth")
    plt.xlabel("Quarter")
    plt.ylabel("GDP Growth (% SAAR)")
    plt.ylim(min(y_real.min(), preds.min()) - 0.5, max(y_real.max(), preds.max()) + 0.5)
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

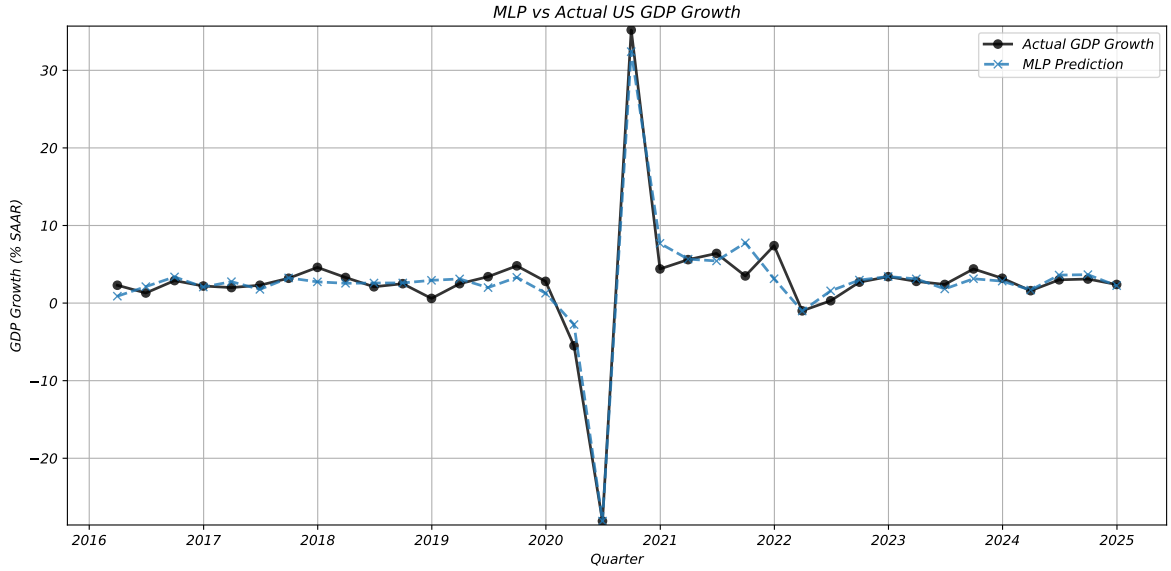

Random Forest vs Actual US GDP Growth
Actual GDP Growth
Random Forest Prediction
GDP Growth (% SAAR)
Quarter
30
20
10
0
−10
−20
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025


Gradient Boosting vs Actual US GDP Growth
Actual GDP Growth
Gradient Boosting Prediction
GDP Growth (% SAAR)
Quarter
30
20
10
0
−10
−20
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025


Ridge vs Actual US GDP Growth
Actual GDP Growth
Ridge Prediction
GDP Growth (% SAAR)
30
20
10
0
−10
−20
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
Quarter


Lasso vs Actual US GDP Growth
Actual GDP Growth
Lasso Prediction
GDP Growth (% SAAR)
30
20
10
0
−10
−20
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
Quarter






```
# AR(1) model on GDP growth
from statsmodels.tsa.ar_model import AutoReg

ar1_model = AutoReg(y_real[y_real.index >= '2016'].dropna(), lags=1).fit()
ar1_pred = ar1_model.fittedvalues
ar1_pred = ar1_pred[ar1_pred.index >= '2016']

plt.figure(figsize=(12, 6))
```

```
plt.plot(y_real.index, y_real, label="Actual GDP Growth", color="black", linewidth=2, marker=
plt.plot(ar1_pred.index, ar1_pred, label="AR(1) Prediction", linestyle="--", color="red", li
plt.title("AR(1) vs Actual US GDP Growth")
plt.xlabel("Quarter")
plt.ylabel("GDP Growth (% SAAR)")
plt.ylim(min(y_real.min(), ar1_pred.min()) - 0.5, max(y_real.max(), ar1_pred.max()) + 0.5)
```

```
(-28.6, 35.7)
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

